

Documentation - Messagerie

Documentation — Messagerie (Chat)

1. Architecture générale

Frontend (Next.js)		Backend (Express)
ChatClient.tsx		
└─ loadConversations	—GET→	/api/conversations
→ messagesController.list		
└─ selectConversation	—GET→	/api/conversations/:id/messages
→ messagesController.getConversationMessages		
└─ sendMessage	—POST→	/api/conversations/:id/messages
→ messagesController.send	→	messagesService

Le controller reste fin (il valide les IDs et délègue), toute la logique (contrôle d'accès, requêtes SQL) est dans `messagesService.js`.

2. Chargement initial (`useEffect` + `loadConversations`)

```
useEffect(() => {
  const storedUser = localStorage.getItem("user");
  if (!storedUser) { router.push("/login"); return; }
  try { setUser(JSON.parse(storedUser)); }
  catch { router.push("/login"); }
}, []);

const loadConversations = useCallback(async () => {
  try {
    const res = await api.get("/api/conversations");
    setConversations(res.data);
  } finally { setLoading(false); }
}, []);
```

Au montage : on relit l'utilisateur stocké dans `localStorage` (posé au login/register). S'il n'y en a pas, redirection vers `/login`. Une fois l'utilisateur connu, un second `useEffect` déclenche `loadConversations()` qui va chercher la liste des conversations de l'utilisateur.

3. `selectConversation` — ouvrir une conversation

```
const selectConversation = useCallback(async (conv) => {
  setSelectedConv(conv);
  setMobileView("detail");
  const res = await
api.get(`/api/conversations/${conv.id}/messages`);
  setMessages(res.data);
  setConversations((prev) =>
    prev.map((c) => (c.id === conv.id ? { ...c, unread_count: 0 }
: c))
```

```
);  
}, []);
```

1. Marque la conversation comme sélectionnée et bascule l’affichage mobile sur la vue détail.
2. Récupère les messages de cette conversation via l’API — **c’est cet appel qui marque aussi les messages comme lus côté backend** (voir section 6).
3. Met à jour localement `unread_count` à 0 dans la liste, pour que le badge de messages non lus disparaisse immédiatement sans attendre un rechargement.

Un `useEffect` séparé lit le paramètre d’URL `?conv=ID` (par exemple un lien “envoyer un message” depuis une autre page) et sélectionne automatiquement la bonne conversation dès que la liste est chargée, une seule fois (`autoSelectedRef`).

4. `sendMessage` — envoyer un message

```
const sendMessage = async () => {  
  const content = newMessage.trim();  
  if (!content || !selectedConv || sending) return;  
  
  setSending(true);  
  try {  
    const res = await  
api.post(`/api/conversations/${selectedConv.id}/messages`, {  
content });  
    setMessages((prev) => [...prev, res.data]);  
    setNewMessage("");  
    setConversations((prev) =>  
      prev.map((c) =>  
        c.id === selectedConv.id  
          ? { ...c, last_message: content, last_message_at: new  
Date().toISOString() }  
          : c  
        )  
      );  
  } finally { setSending(false); }  
};
```

1. Ignore l’envoi si le champ est vide, si aucune conversation n’est sélectionnée, ou si un envoi est déjà en cours (évite le double-clic).
 2. Envoie le message au backend, puis **ajoute directement la réponse du serveur** (`res.data` , qui contient déjà `sender_name` / `sender_picture`) à la liste des messages affichés — pas besoin de recharger toute la conversation.
 3. Met à jour l’aperçu (`last_message`) dans la sidebar, pour un retour visuel immédiat.
 4. `handleKeyDown` permet d’envoyer avec la touche Entrée (sans Shift, pour garder Shift+Entrée disponible pour un retour à la ligne).
-

5. Petits détails d’UI notables

- `ChatSkeleton` : écran de chargement avec des blocs gris animés (`animate-pulse`), affiché tant que `loading` est vrai, pour éviter un simple texte “Chargement...” et limiter le décalage visuel (CLS) à l’arrivée des vraies données.
 - `ConversationItem` : extrait en composant séparé et cadré autour de `conv` / `isSelected` / `onSelect` , pour éviter de recréer toutes les lignes de la liste à chaque frappe dans le champ de message.
 - Séparateurs de date (`isSameDay` / `formatDateSeparator`) : affichés entre deux messages seulement si le jour a changé, comme dans une app de messagerie classique.
-

6. Backend — contrôle d'accès (`assertParticipant`)

```
async function assertParticipant(db, conversationId, userId) {
  const conv = await db.getAsync('SELECT id, user1_id, user2_id
FROM conversations WHERE id = ?', [conversationId]);
  if (!conv) throw 404;
  if (conv.user1_id !== userId && conv.user2_id !== userId) throw
403;
  return conv;
}
```

Cette fonction est appelée **avant** de lire ou d'envoyer des messages (`getMessages`, `sendMessage`). Elle vérifie que l'utilisateur connecté fait bien partie des deux participants de la conversation — sinon `403 Forbidden`. C'est le verrou de sécurité central de toute la messagerie : sans lui, n'importe quel utilisateur authentifié pourrait lire les messages de n'importe quelle conversation en devinant son ID.

7. Backend — créer ou récupérer une conversation (`getOrCreateConversation`)

- Convertit les deux IDs en nombres, rejette si l'un des deux n'est pas un entier valide.
 - Interdit de démarrer une conversation avec soi-même.
 - Vérifie que le destinataire existe en base.
 - Cherche une conversation existante entre les deux utilisateurs, **dans les deux sens** (`user1/user2` ou `user2/user1`) — pour ne jamais créer deux conversations en double entre les mêmes personnes.
 - Si aucune n'existe, en crée une nouvelle.
-

8. Backend — lecture des messages (`getMessages`)

1. Vérifie l'accès avec `assertParticipant`.
2. **Marque comme lus** tous les messages reçus (pas envoyés par l'utilisateur courant) qui ne l'étaient pas encore (`UPDATE ... SET read_at = CURRENT_TIMESTAMP`).
3. Renvoie tous les messages de la conversation, triés du plus ancien au plus récent, avec le nom/photo de l'expéditeur joints.

C'est cette étape 2 qui fait disparaître le badge "non lu" côté frontend dès qu'on ouvre la conversation.

9. Backend — envoi d'un message (`sendMessage`)

1. Vérifie l'accès avec `assertParticipant`.
2. Valide le contenu : non vide après `trim()`, et limité à 5000 caractères max.
3. Insère le message, puis met à jour `updated_at` sur la conversation (sert au tri par activité récente dans `listConversations`).
4. Renvoie le message créé avec les infos de l'expéditeur jointes — c'est cet objet complet que le frontend ajoute directement à sa liste locale.

`startConversation` combine `getOrCreateConversation` + `sendMessage` en un seul appel, pratique pour un bouton "Envoyer un message" depuis un profil ou une propriété, sans devoir faire deux requêtes.

10. Backend — liste des conversations (`listConversations`)

Une seule requête SQL avec des sous-requêtes pour récupérer, pour chaque conversation : l'identité de "l'autre" utilisateur (`CASE WHEN user1_id = ? THEN user2_id ELSE user1_id END`), le dernier message envoyé, sa date, et le nombre de messages non lus envoyés par l'autre personne. Le tri se fait par activité la plus récente (dernier message, ou date de mise à jour si aucun message).

11. Résumé express

- **Frontend** : charge les conversations au montage → sélection d'une conversation = fetch de ses messages (qui les marque lus côté serveur) → envoi d'un message = POST puis ajout optimiste de la réponse à la liste locale.
- **Sécurité** : `assertParticipant` bloque tout accès à une conversation dont l'utilisateur ne fait pas partie (403), sur la lecture comme sur l'envoi.
- **Anti-doublon** : une conversation entre deux utilisateurs est unique, recherchée dans les deux sens avant d'en créer une nouvelle.
- **Validation** : contenu du message non vide, limité à 5000 caractères ; IDs toujours validés comme entiers positifs côté controller.